REVISION CHEAT SHEET:

Collections in C#

TOPIC 21 · C#

Interview-ready notes · examples · C# · diagrams

Topic 21 of 40

Collections are among the most frequently used parts of C# and .NET.

A collection lets you store and work with:

EXAMPLE / FLOW

```
Zero, one, or many values
```

instead of creating separate variables:

C#

```
Gift gift1;
Gift gift2;
Gift gift3;
```

you can write:

C#

```
List<Gift> gifts = [];
```

and dynamically store:

EXAMPLE / FLOW

```
Gift 1
Gift 2
Gift 3
...
Gift N
```

The important question is not merely:

> “How do I store multiple objects?”

It is:

> **What operations matter most—indexing, uniqueness, key lookup, ordering, FIFO/LIFO processing, sorted access, mutation, or read-only exposure?**

.NET provides specialized collection types for these different requirements. Microsoft recommends generic collections for strongly typed application code, while concurrent collections are intended for multi-threaded mutation scenarios.

High-Level Mental Model

EXAMPLE / FLOW

```
Need multiple values
        ↓
What operation matters?
```

EXAMPLE / FLOW

```
Ordered sequence + index
        →
List<T>
```

EXAMPLE / FLOW

```
Key → Value lookup
        →
Dictionary<TKey, TValue>
```

EXAMPLE / FLOW

```
Unique values
        →
HashSet<T>
```

EXAMPLE / FLOW

```
First-In-First-Out
        →
Queue<T>
```

EXAMPLE / FLOW

```
Last-In-First-Out
        →
Stack<T>
```

EXAMPLE / FLOW

```
Always sorted
        →
SortedSet<T>
SortedDictionary<TKey,TValue>
SortedList<TKey,TValue>
```

Fundraising Examples

EXAMPLE / FLOW

```
Gift.Allocations
    →
List<GiftAllocation>
```

EXAMPLE / FLOW

```
GiftId → Gift
    →
Dictionary<long, Gift>
```

EXAMPLE / FLOW

```
Unique Fund IDs
    →
HashSet<long>
```


EXAMPLE / FLOW

```
Receipts waiting to send
    →
Queue<GiftReceipt>
```

EXAMPLE / FLOW

```
Undo processing steps
    →
Stack<GiftChange>
```

Topic Coverage

This topic covers:

- what collections are;
- generic vs non-generic collections;
- array vs collection;
- collection interface hierarchy;
- `IEnumerable<T>`;
- `ICollection<T>`;
- `IList<T>`;
- `IReadOnlyCollection<T>`;
- `IReadOnlyList<T>`;
- `List<T>`;
- count vs capacity;
- collection expressions;
- adding/removing/searching/sorting;
- mutation during enumeration;
- `Dictionary<TKey,TValue>`;
- dictionary key equality;
- `TryGetValue`;
- `TryAdd`;
- `HashSet<T>`;
- mathematical set operations;
- equality comparers;
- `Queue<T>`;
- `Stack<T>`;
- `LinkedList<T>`;
- `SortedSet<T>`;
- `SortedDictionary<TKey,TValue>`;
- `SortedList<TKey,TValue>`;
- choosing collections;
- complexity;
- memory/performance;
- read-only vs immutable;

- thread safety;
- ASP.NET Core usage;
- Fundraising examples;
- common mistakes;
- practical exercises;
- interview questions.

1. What Is a Collection?

A collection is an object that stores multiple elements.

Example:

```C#
List<decimal> giftAmounts = [100m, 250m, 500m];
```

The collection contains:

EXAMPLE / FLOW

```
100
250
500
```

Every element is strongly typed as:

EXAMPLE / FLOW

```
decimal
```

Collections vs Individual Variables

Without a collection:

```C#
decimal amount1 = 100m;
decimal amount2 = 250m;
decimal amount3 = 500m;
```

This becomes impractical when the number of elements changes dynamically.

With:

```C#
List<decimal> amounts = [];
```

you can dynamically:

EXAMPLE / FLOW

```
Add

Remove

Search

Iterate

Sort
```

elements.

2. Generic vs Non-Generic Collections

Older .NET APIs include non-generic types from:

```
C#
System.Collections
```

such as:

EXAMPLE / FLOW

```
ArrayList

Hashtable

Queue

Stack
```

Modern application code usually prefers:

```
C#
System.Collections.Generic
```

such as:

EXAMPLE / FLOW

```
List<T>

Dictionary<TKey,TValue>

HashSet<T>

Queue<T>

Stack<T>
```

Generic collections provide compile-time type safety and avoid much of the casting and value-type boxing associated with old object-based collections.

Old ArrayList Example

```
C#
ArrayList values = new();

values.Add(100);
values.Add("Gift");
values.Add(DateTime.UtcNow);
```

Everything is effectively handled as:

```
C#
object
```

You can accidentally mix unrelated types.

Then:

```C#
int amount = (int)values[0];
```

requires casting.

Generic List

```C#
List<int> amounts = [];

amounts.Add(100);
amounts.Add(250);
```

This is rejected:

```C#
// amounts.Add("Gift");
```

because the compiler knows:

EXAMPLE / FLOW

```
List<int>
```

accepts:

EXAMPLE / FLOW

```
int
```

only.

3. Arrays vs Collections

You already studied arrays in Topic 10.

Array:

```C#
decimal[] amounts = [100m, 250m, 500m];
```

`List<T>`:

```C#
List<decimal> amounts = [100m, 250m, 500m];
```

They look similar, but their size behavior differs.

Array

EXAMPLE / FLOW

```
Fixed Length
```

After creation:

```C#
decimal[] values = new decimal[3];
```

its length remains:

EXAMPLE / FLOW

```
3
```

You cannot append a fourth element directly.

List

EXAMPLE / FLOW

```
Dynamic Logical Size
```

C#

```
List<decimal> values = [];

values.Add(100m);
values.Add(250m);
values.Add(500m);
values.Add(1000m);
```

The list expands its backing storage when required.

Array vs List

Feature	Array	List<T>
Size	Fixed	Dynamic
Indexed access	Yes	Yes
Add/remove API	No	Yes
Contiguous storage	Yes	Backed by array
Length/count	`Length`	`Count`
Capacity	Fixed length	Separate capacity
Best for	Fixed known data	Dynamic ordered collection

4. Important Collection Interfaces

The most important sequence interfaces are:

EXAMPLE / FLOW

```
IEnumerable<T>
    ↓
ICollection<T>
    ↓
IList<T>
```

The exact inheritance model also includes:

EXAMPLE / FLOW

```
IReadOnlyCollection<T>

IReadOnlyList<T>
```

and specialized families:

EXAMPLE / FLOW

```
IDictionary<TKey,TValue>

ISet<T>
```

Mental Model

EXAMPLE / FLOW

```
IEnumerable<T>
    →
Can enumerate
```

EXAMPLE / FLOW

```
ICollection<T>
    →
Can enumerate
+
Count
+
Add / Remove / Contains
```

EXAMPLE / FLOW

```
IList<T>
    →
Collection
+
Ordered index access
+
Insert / RemoveAt
```

5. IEnumerable<T>

Definition

`IEnumerable<T>` represents something that can be enumerated.

Conceptually:

EXAMPLE / FLOW

```
Give me elements
one after another.
```

Example:

C#

```
IEnumerable<Gift> gifts = GetGifts();
```

You can:

C#

```
foreach (
    Gift gift
    in gifts)
{
    Console.WriteLine(
        gift.GiftId);
}
```

IEnumerable Does Not Mean List

An `IEnumerable<T>` might be backed by:

EXAMPLE / FLOW

```
Array

List<T>

HashSet<T>

Dictionary projection

Iterator

Database query

Generated sequence
```

Therefore don't assume:

C#

```
gifts[0]
```

exists merely because something is `IEnumerable<Gift>`.

IEnumerable Contract

The core promise is approximately:

EXAMPLE / FLOW

```
This sequence can be enumerated.
```

It does not inherently promise:

EXAMPLE / FLOW

```
Index access

Mutation

Count

Ordering

Materialization
```

Why Return IEnumerable?

Suppose a caller only needs:

EXAMPLE / FLOW

```
foreach
```

Then:

C#

```
IEnumerable<Gift>
```

may expose exactly enough capability.

This follows an important API principle:

> **Expose the least powerful abstraction that satisfies the consumer.**

6. ICollection<T>

`ICollection<T>` extends the idea of an enumerable collection with operations such as:

```
EXAMPLE / FLOW
Count

Add

Remove

Contains

Clear

CopyTo
```

Microsoft's interface definition includes these mutation and collection-size operations.

Example:

```
C#
ICollection<GiftAllocation>
    allocations = new List<GiftAllocation>();
```

ICollection Mental Model

```
EXAMPLE / FLOW
IEnumerable<T>
    +
How many?
    +
Add
    +
Remove
    +
Contains
```

But it does not require:

```
EXAMPLE / FLOW
Index 0
Index 1
Index 2
```

7. IList<T>

`IList<T>` represents an ordered collection with index-based operations.

Example:

```
C#
IList<Gift> gifts = new List<Gift>();

gifts.Add(
    gift);

Gift first = gifts[0];
```

It supports concepts such as:

```
EXAMPLE / FLOW
Indexer

Insert

RemoveAt

IndexOf
```

List Mental Model

```
EXAMPLE / FLOW
Order matters
+
Position matters
+
Duplicates are allowed
```

Example:

```
C#
List<string> values = ["Gift", "Gift", "Campaign"];
```

Duplicates are perfectly valid.

8. Read-Only Collection Interfaces

Two very useful interfaces are:

```
C#
IReadOnlyCollection<T>
```

and:

```
C#
IReadOnlyList<T>
```

IReadOnlyCollection

Provides:

```
EXAMPLE / FLOW
Enumeration

Count
```

without exposing normal mutation operations.

IReadOnlyList

Adds:

EXAMPLE / FLOW

```
Indexed read access
```

Example:

C#

```
IReadOnlyList<GiftAllocation>
    Allocations
        =>
        _allocations;
```

Microsoft notes that `IReadOnlyList<T>` makes the list's number and ordering read-only through that interface, but the element objects themselves are not guaranteed to be immutable.

Important: Read-Only ≠ Immutable

Suppose:

C#

```
private readonly List<GiftAllocation>
    _allocations = [];

public IReadOnlyList<GiftAllocation>
    Allocations =>
    _allocations;
```

The caller cannot write:

C#

```
// gift.Allocations.Add(...)
```

through `IReadOnlyList<T>`.

But:

C#

```
_allocations.Add(...)
```

inside `Gift` still works.

And Elements May Still Mutate

If:

C#

```
GiftAllocation allocation = gift.Allocations[0];
```

and allocation has mutable state:

C#

```
allocation.Amount = 500m;
```

then:

has **not** made:

EXAMPLE / FLOW
GiftAllocation

immutable.

Read-Only vs Immutable

EXAMPLE / FLOW
Read-only interface
→
Consumer can't mutate collection
through this reference

EXAMPLE / FLOW
Immutable collection
→
Collection itself produces new
versions rather than mutating

Immutable collections will be covered more deeply in Topic 29.

9. List<T>

`List<T>` is probably the most frequently used general-purpose mutable collection.

Use it when you need:

EXAMPLE / FLOW
Ordered sequence

Duplicates

Dynamic size

Fast indexed access

Efficient append

Microsoft's current collection complexity guidance lists indexed access as O(1) and `Add` as amortized O(1), with occasional O(n) resizing.

Creating a List

Classic:

C#
```
List<Gift> gifts = new List<Gift>();
```

Target-typed:

C#
```
List<Gift> gifts = new();
```

Modern collection expression:

```C#
List<Gift> gifts = [];
```

Collection expressions use:

```C#
[ ... ]
```

and can target many collection types, including `List<T>` and several collection interfaces.

Collection Expression with Values

```C#
List<decimal> amounts = [100m, 250m, 500m];
```

Spread existing values:

```C#
decimal[] historical = [50m, 75m];

List<decimal> all = [.. historical, 100m, 250m];
```

The:

```C#
..
```

inside a collection expression spreads the elements of another enumerable into the new collection.

List Add

```C#
gifts.Add(
    gift);
```

Add multiple:

```C#
gifts.AddRange(
    otherGifts);
```

Index Access

```C#
Gift first = gifts[0];
```

Last:

```C#
Gift last = gifts[^1];
```

Invalid indexes produce:

```
EXAMPLE / FLOW
ArgumentOutOfRangeException
```

for `List<T>` index operations.

Count

```
C#
int count = gifts.Count;
```

For a list:

```
EXAMPLE / FLOW
Count
```

means:

```
EXAMPLE / FLOW
How many actual elements
are currently stored?
```

Count vs Capacity

This distinction matters.

```
C#
List<Gift> gifts = new(capacity: 100);
```

Initially:

```
EXAMPLE / FLOW
Count = 0

Capacity >= 100
```

The list has room for elements but contains none yet.

Mental Model

```
EXAMPLE / FLOW
Capacity
    →
Internal storage available
```

```
EXAMPLE / FLOW
Count
    →
Actual logical elements
```

Capacity Growth

When:

and another element is added, the list must grow its internal storage.

That can require:

```
EXAMPLE / FLOW
Allocate larger backing array

Copy existing elements
```

which explains why `List<T>.Add` is:

```
EXAMPLE / FLOW
O(1) amortized
```

but can occasionally become:

```
EXAMPLE / FLOW
O(n)
```

during growth.

Preallocating Capacity

If you already know approximately how many items are coming:

```
C#
List<Gift> gifts = new(expectedCount);
```

can reduce resizing.

But don't blindly allocate huge capacities from untrusted input.

The same principle applies to hash-based collection capacity; Microsoft specifically cautions against directly trusting user-controlled capacity values because excessive allocation can become a resource problem.

Insert

```
C#
gifts.Insert(
    0,
    gift);
```

Inserting near the beginning generally requires shifting later elements.

Therefore insertion in the middle/beginning is:

```
EXAMPLE / FLOW
O(n)
```

for an array-backed list.

Remove

Remove first matching value:

```C#
bool removed = gifts.Remove(gift);
```

Remove by index:

```C#
gifts.RemoveAt(
    0);
```

Remove range:

```C#
gifts.RemoveRange(
    index: 0,
    count: 5);
```

RemoveAll

```C#
int removed = gifts.RemoveAll(gift => gift.Status == GiftStatus.Cancelled);
```

This uses a:

```C#
Predicate<Gift>
```

which connects directly to Topic 22.

Contains

```C#
bool exists = gifts.Contains(gift);
```

Because a list isn't hash-indexed, this generally requires scanning elements:

EXAMPLE / FLOW

```
O(n)
```

IndexOf

```C#
int index = gifts.IndexOf(gift);
```

Returns:

EXAMPLE / FLOW

```
Index
```

or:

EXAMPLE / FLOW

```
-1
```

if not found.

Sort

```C#
amounts.Sort();
```

For custom objects:

```C#
gifts.Sort(
    (left, right) =>
        left.TotalAmount.CompareTo(
            right.TotalAmount));
```

Delegates/lambdas become Topic 22.

Reverse

```C#
gifts.Reverse();
```

This mutates the list order.

Clear

```C#
gifts.Clear();
```

After:

EXAMPLE / FLOW

```
Count = 0
```

Capacity may remain available for reuse.

List Performance Summary

Typical complexity:

Operation	Typical Complexity
`list[index]`	O(1)
`Add`	Amortized O(1)
`Insert` middle	O(n)
`Remove` by value	O(n)
`RemoveAt` middle	O(n)
`Contains`	O(n)
Enumeration	O(n)
Sorting	O(n log n) typical

Microsoft's collection guidance specifically documents O(1) indexed access and amortized O(1) append for `List<T>`.

10. Enumeration and Mutation

Typical:

```C#
foreach (
    Gift gift
    in gifts)
{
    Console.WriteLine(
        gift.GiftId);
}
```

Do not structurally modify most mutable collections while enumerating them.

Bad:

```C#
foreach (
    Gift gift
    in gifts)
{
    if (
        gift.Status
        == GiftStatus.Cancelled) { gifts.Remove(gift);
    }
}
```

This commonly results in:

EXAMPLE / FLOW

```
InvalidOperationException
```

because the collection changed during enumeration.

Better List Removal

```C#
gifts.RemoveAll(
    gift =>
        gift.Status
        == GiftStatus.Cancelled);
```

Or iterate backward:

```C#
for (
    int i = gifts.Count - 1;
    i >= 0;
    i--)
{
    if (
        gifts[i].Status
        == GiftStatus.Cancelled) { gifts.RemoveAt(i);
    }
}
```

Why Backward?

If you remove from the end toward the beginning:

```
EXAMPLE / FLOW
Removing index i
```

doesn't invalidate indexes you still plan to visit.

11. Dictionary<TKey,TValue>

A dictionary maps:

```
EXAMPLE / FLOW
Key
    →
Value
```

Example:

```
C#
Dictionary<long, Gift> giftsById = new();
```

Add:

```
C#
giftsById.Add(
    gift.GiftId,
    gift);
```

Lookup:

```
C#
Gift gift = giftsById[1001];
```

`Dictionary<TKey,TValue>` is implemented as a hash table; key lookup is typically close to O(1), although hash quality and collisions matter. Keys must be unique under the dictionary's equality comparer.

Dictionary Mental Model

```
EXAMPLE / FLOW
Key
must be unique
```

while:

```
EXAMPLE / FLOW
Values
may repeat
```

Example:

```
EXAMPLE / FLOW
1001 → Gift A

1002 → Gift B
```

```
1003 → Gift C
```


Duplicate Key

```C#
giftsById.Add(
    1001,
    giftA);

giftsById.Add(
    1001,
    giftB);
```

throws because:

EXAMPLE / FLOW

```
1001
```

already exists.

Dictionary Indexer

```C#
giftsById[
    1001] = gift;
```

The indexer behaves differently from `Add`.

If key doesn't exist:

EXAMPLE / FLOW

```
Adds key/value
```

If key exists:

EXAMPLE / FLOW

```
Replaces value
```

Lookup with Indexer

```C#
Gift gift = giftsById[giftId];
```

If `giftId` is absent:

EXAMPLE / FLOW

```
KeyNotFoundException
```

Prefer TryGetValue When Absence Is Normal

```C#
if (
    giftsById.TryGetValue(
        giftId,
```

```
        out Gift? gift))
{
    Console.WriteLine(
        gift.TotalAmount);
}
```

Else:

EXAMPLE / FLOW

```
Key not present
```

without exception-based control flow.

ContainsKey

C#

```
if (
    giftsById.ContainsKey(
        giftId))
{
    Gift gift = giftsById[giftId];
}
```

This works.

But:

EXAMPLE / FLOW

```
ContainsKey
+
Indexer
```

can perform two lookups.

If you need the value:

C#

```
TryGetValue
```

is usually preferable.

TryAdd

C#

```
bool added = giftsById.TryAdd(gift.GiftId, gift);
```

If the key already exists:

EXAMPLE / FLOW

```
false
```

rather than throwing.

Dictionary Enumeration

C#

```
foreach (
    KeyValuePair<long, Gift> pair
```

```
    in giftsById)
{
    Console.WriteLine(
        $"{pair.Key}: {pair.Value.TotalAmount}");
}
```

Each dictionary element is a:

```C#
KeyValuePair<TKey,TValue>
```

which Microsoft documents as the element exposed during dictionary enumeration.

Deconstruction

Modern C# supports:

```C#
foreach (
    (long giftId, Gift gift)
    in giftsById)
{
    Console.WriteLine(
        giftId);
}
```

Keys

```C#
Dictionary<long, Gift>.KeyCollection
    keys = giftsById.Keys;
```

Typically you simply write:

```C#
foreach (
    long id
    in giftsById.Keys)
{
}
```

Values

```C#
foreach (
    Gift gift
    in giftsById.Values)
{
}
```

Dictionary Key Equality

The dictionary must determine:

EXAMPLE / FLOW

```
Are these two keys the same?
```

By default it uses:

```C#
EqualityComparer<TKey>.Default
```

unless you provide another comparer.

String Key Example

Case-sensitive:

```C#
Dictionary<string, Gift> byReference = new();
```

By default:

```EXAMPLE / FLOW
"GFT-1001"
```

and:

```EXAMPLE / FLOW
"gft-1001"
```

are different strings.

Case-Insensitive Dictionary

```C#
Dictionary<string, Gift> byReference = new(StringComparer.OrdinalIgnoreCase);
```

Now:

```EXAMPLE / FLOW
"GFT-1001"

"gft-1001"
```

are treated as the same key.

Microsoft explicitly recommends supplying an appropriate `IEqualityComparer<TKey>` when custom key equality is required.

Do Not Mutate Dictionary Keys

Suppose key is a mutable object whose:

```EXAMPLE / FLOW
Equals
```

or:

```EXAMPLE / FLOW
GetHashCode
```

changes after insertion.

Then the dictionary may no longer be able to locate the item correctly.

Microsoft specifically warns that a dictionary key must not change in a way that affects while being used as a key.

Good Dictionary Keys

Often:

EXAMPLE / FLOW

```
int

long

Guid

string

immutable record/value object
```

make sensible keys.

Bad Mutable Key

C#

```
public sealed class MutableGiftKey
{
    public long Id
    {
        get;
        set;
    }

    public override int GetHashCode()
    {
        return Id.GetHashCode();
    }
}
```

Insert:

C#

```
key.Id = 1001;

dictionary[key] = gift;
```

Then:

C#

```
key.Id = 5000;
```

The key's hash identity has changed.

Avoid this.

Dictionary Complexity

Typical:

Operation	Average
Add	O(1)
Lookup	O(1)
Remove	O(1)
ContainsKey	O(1)
Enumeration	O(n)

Hash collisions can degrade behavior, so these are typical/average expectations rather than universal guarantees. Microsoft's current complexity table lists average O(1) dictionary add/lookup behavior.

12. HashSet<T>

A set stores:

EXAMPLE / FLOW

```
Unique elements
```

Example:

C#

```
HashSet<long> fundIds = [10, 20, 30];
```

Try duplicate:

C#

```
bool added = fundIds.Add(20);
```

Result:

EXAMPLE / FLOW

```
false
```

The set remains:

EXAMPLE / FLOW

```
10
20
30
```

`HashSet<T>` is an unordered set designed for uniqueness and efficient set operations.

HashSet vs List

List:

EXAMPLE / FLOW

```
Duplicates allowed

Order/position important
```

HashSet:

EXAMPLE / FLOW

```
No duplicates

Membership/uniqueness important
```

Common Fundraising Example

Suppose a gift must not contain duplicate fund designations.

C#

```
HashSet<long> allocatedFundIds = new();
```

Then:

C#

```
if (
    !allocatedFundIds.Add(
        fundId))
{
    throw new InvalidOperationException(
        "Fund already allocated.");
}
```

`Add` does both:

EXAMPLE / FLOW

```
Try inserting
+
Tell whether value was new
```

Contains

C#

```
bool exists = allocatedFundIds.Contains(fundId);
```

Hash membership is typically:

EXAMPLE / FLOW

```
O(1) average
```

rather than a linear list scan.

HashSet Set Operations

`HashSet<T>` directly supports mathematical set operations including:

EXAMPLE / FLOW

```
Union

Intersection

Difference

Symmetric difference
```

through methods such as `UnionWith`, `IntersectWith`, `ExceptWith`, and `SymmetricExceptWith`.

Union

```C#
HashSet<long> first = [1, 2];

HashSet<long> second = [2, 3];

first.UnionWith(
    second);
```

Result:

```EXAMPLE / FLOW
1
2
3
```

Intersection

```C#
first.IntersectWith(
    second);
```

Keeps only elements present in both sets.

Difference

```C#
first.ExceptWith(
    second);
```

Removes values found in:

```EXAMPLE / FLOW
second
```

Set Relationships

Useful methods include:

```C#
IsSubsetOf

IsSupersetOf

Overlaps

SetEquals
```

These can model business rules elegantly.

Example

Allowed funds:

```C#
HashSet<long> allowed = [100, 200, 300];
```

Requested funds:

```C#
HashSet<long> requested = [100, 300];
```

Check:

```C#
bool valid = requested.IsSubsetOf(allowed);
```

HashSet Equality Comparer

Just like dictionary keys:

```C#
HashSet<string> references = new(StringComparer.OrdinalIgnoreCase);
```

Then duplicates differing only by casing are rejected.

13. Queue<T>

A queue follows:

```EXAMPLE / FLOW
FIFO
```

meaning:

```EXAMPLE / FLOW
First In
First Out
```

Microsoft describes `Queue<T>` as a FIFO collection and documents its principal operations as `Enqueue`, `Dequeue`, and `Peek`.

Queue Mental Model

Imagine donors waiting in line:

```EXAMPLE / FLOW
A
B
C
```

A entered first.

Therefore A leaves first.

Queue Operations

```C#
Queue<Gift> pending = new();
```

Add:

```C#
pending.Enqueue(
    gift1);

pending.Enqueue(
    gift2);
```

Remove oldest:

```C#
Gift gift = pending.Dequeue();
```

Inspect without removing:

```C#
Gift next = pending.Peek();
```

Safer Try APIs

```C#
if (
    pending.TryDequeue(
        out Gift? gift))
{
    Process(
        gift);
}
```

And:

```C#
if (
    pending.TryPeek(
        out Gift? next))
{
}
```

These avoid exceptions when an empty queue is a normal possibility.

Fundraising Use Case

```EXAMPLE / FLOW
Receipt sending queue

Import rows

Gift reconciliation work

Pending notifications
```

Example:

```EXAMPLE / FLOW
Gift A enters

Gift B enters
```

```
Gift C enters

Process order:
A → B → C
```

Queue Performance

`Queue<T>` uses a circular-array implementation. Enqueue is usually O(1) and can occasionally become O(n) when internal capacity grows; dequeue is O(1).

14. Stack<T>

A stack follows:

EXAMPLE / FLOW

```
LIFO
```

meaning:

EXAMPLE / FLOW

```
Last In
First Out
```

Microsoft describes `Stack<T>` as an array-backed LIFO collection with `Push`, `Pop`, and `Peek` as its core operations.

Stack Mental Model

EXAMPLE / FLOW

```
Top
 ↓
C
B
A
```

C was added last.

Therefore C is removed first.

Push

C#

```
Stack<string> history = new();

history.Push(
    "Created");

history.Push(
    "Validated");

history.Push(
    "Completed");
```

Pop

```C#
string last = history.Pop();
```

Returns:

```
EXAMPLE / FLOW
Completed
```

Peek

```C#
string current = history.Peek();
```

returns the top element without removing it.

TryPop

```C#
if (
    history.TryPop(
        out string? state))
{
}
```

Common Stack Uses

```
EXAMPLE / FLOW
Undo

Backtracking

Expression parsing

Tree traversal

Call-history-like processing
```

Fundraising example:

```
EXAMPLE / FLOW
Undo Gift editing steps
```

Stack Performance

Microsoft documents `Pop` as O(1), while `Push` is normally O(1) but may become O(n) when capacity must grow.

Queue vs Stack

Collection	Ordering
`Queue<T>`	FIFO

Collection	Ordering
`Stack<T>`	LIFO

Think:

EXAMPLE / FLOW

```
Queue
    →
Waiting line
```

EXAMPLE / FLOW

```
Stack
    →
Stack of plates
```

15. LinkedList<T>

`LinkedList<T>` represents a doubly linked sequence.

Conceptually:

EXAMPLE / FLOW

```
Node A
 ↕
Node B
 ↕
Node C
```

Each node knows:

EXAMPLE / FLOW

```
Previous

Next
```

When LinkedList Can Help

It can be useful when:

EXAMPLE / FLOW

```
You already have a node reference

Frequent insertion/removal
occurs around known nodes

Random indexing is not required
```

Example:

C#

```
LinkedList<string> workflow = new();

LinkedListNode<string> first = workflow.AddFirst("Validate");

workflow.AddAfter(
    first,
    "Persist");
```

Important Limitation

A linked list does **not** provide:

```
C#
list[100]
```

like `List<T>`.

Finding the 100th element requires traversal.

For most general application sequences:

```
EXAMPLE / FLOW
List<T>
```

is usually the simpler default.

16. Sorted Collections

Sometimes your collection must remain sorted automatically.

Main options include:

```
EXAMPLE / FLOW
SortedSet<T>

SortedDictionary<TKey,TValue>

SortedList<TKey,TValue>
```

SortedSet<T>

Use when you need:

```
EXAMPLE / FLOW
Unique elements

Automatically sorted
```

Example:

```
C#
SortedSet<decimal> amounts = [500m, 100m, 250m];
```

Enumeration produces sorted order:

```
EXAMPLE / FLOW
100
250
500
```

HashSet vs SortedSet

```
EXAMPLE / FLOW
HashSet<T>
    →
Faster typical hash lookup
```

```
+
No guaranteed sorted order
```

EXAMPLE / FLOW

```
SortedSet<T>
    →
Sorted
+
Tree-based
+
O(log n) operations
```

Microsoft's current collection complexity table lists `HashSet<T>` add/lookup as amortized O(1) and `SortedSet<T>.Add` as O(log n).

SortedDictionary<TKey,TValue>

Dictionary semantics:

EXAMPLE / FLOW

```
Key
    →
Value
```

but keys remain sorted.

Example:

C#

```
SortedDictionary<DateOnly, decimal>
    dailyTotals = new();
```

SortedDictionary Complexity

Microsoft documents `SortedDictionary<TKey,TValue>` as a binary search tree with O(log n) retrieval and O(log n) normal insertion/removal behavior for unsorted incoming data.

SortedList<TKey,TValue>

Despite its name, it is also:

EXAMPLE / FLOW

```
Key → Value
```

rather than an ordinary:

C#

```
List<T>
```

It stores sorted key/value pairs using array-oriented storage.

Microsoft notes that it generally uses less memory than `SortedDictionary`, offers O(log n) retrieval, but insertion/removal can cost O(n) because elements must be shifted.

SortedDictionary vs SortedList

Requirement	Better Candidate
Many inserts/removes	`SortedDictionary`
More compact sorted map	`SortedList`
Indexed access to sorted keys/values useful	`SortedList`
Tree-based dynamic map	`SortedDictionary`

Do not choose based only on the word:

EXAMPLE / FLOW

```
List
```

in `SortedList`.

17. Choosing the Correct Collection

This is one of the most important skills in this topic.

Need Ordered Values?

EXAMPLE / FLOW

```
List<T>
```

Example:

EXAMPLE / FLOW

```
GiftAllocations in sequence
```

Need Unique Values?

EXAMPLE / FLOW

```
HashSet<T>
```

Example:

EXAMPLE / FLOW

```
Unique FundIds
```

Need Value by Key?

EXAMPLE / FLOW

```
Dictionary<TKey,TValue>
```

Example:

EXAMPLE / FLOW

```
GiftId → Gift
```

Need Processing Order?

FIFO:

EXAMPLE / FLOW
```
Queue<T>
```

LIFO:

EXAMPLE / FLOW
```
Stack<T>
```

Need Sorted Unique Values?

EXAMPLE / FLOW
```
SortedSet<T>
```

Need Sorted Key/Value Map?

EXAMPLE / FLOW
```
SortedDictionary<TKey,TValue>
```

or:

EXAMPLE / FLOW
```
SortedList<TKey,TValue>
```

depending on update patterns.

Need Only Enumeration?

Expose:

C#
```
IEnumerable<T>
```

instead of:

C#
```
List<T>
```

when callers should not depend on indexing/mutation.

Need Count + Read Access?

C#
```
IReadOnlyCollection<T>
```

Need Count + Index Read Access?

C#
```
IReadOnlyList<T>
```

Collection Decision Flow

EXAMPLE / FLOW

```
Need many values
    ↓
Need lookup by key?
    ├─ Yes → Dictionary
    └─ No
        ↓
Need uniqueness?
    ├─ Yes
    │    ↓
    │ Need sorted?
    │    ├─ Yes → SortedSet
    │    └─ No  → HashSet
    │
    └─ No
        ↓
Need FIFO?
    ├─ Yes → Queue
    └─ No
        ↓
Need LIFO?
    ├─ Yes → Stack
    └─ No
        ↓
Need dynamic indexed sequence?
        →
       List<T>
```

18. Equality in Collections

Collections need equality for operations such as:

EXAMPLE / FLOW

```
Contains

Remove

Dictionary key matching

HashSet uniqueness
```

The default mechanism generally relies on:

C#

```
EqualityComparer<T>.Default
```

where appropriate.

Value Type Example

C#

```
HashSet<int> values = [1, 2, 3];
```

Equality is integer value equality.

Record Example

```C#
public readonly record struct FundId(
    long Value);
```

Then:

```C#
HashSet<FundId> funds = [new(100), new(100)];
```

contains one logical value because record-struct value equality considers them equal.

Class Entity Example

For a normal class that doesn't override equality:

```C#
Gift first = new(...);

Gift second = new(...);
```

even if they contain the same business data, default reference equality may treat them as different objects.

This matters when using entities inside:

```C#
HashSet<Gift>
```

Better Entity Set Key

Often you really want:

```C#
HashSet<long> giftIds
```

rather than relying on entity-object equality.

IEqualityComparer

If equality rules differ from default behavior, provide a comparer.

Example:

```C#
HashSet<string> campaignCodes = new(StringComparer.OrdinalIgnoreCase);
```

Equality Must Match Hashing

When implementing custom equality for hash-based collections:

EXAMPLE / FLOW

```
Equal values
```

must produce compatible:

results.

Otherwise:

EXAMPLE / FLOW

```
Dictionary

HashSet
```

can behave incorrectly.

19. Collection Expressions in Modern C#

Modern syntax:

```C#
List<long> ids = [1, 2, 3];
```

Array:

```C#
long[] ids = [1, 2, 3];
```

Interface target:

```C#
IReadOnlyList<long> ids = [1, 2, 3];
```

Microsoft documents collection expressions as target-typed syntax supported by arrays, lists, spans, and several generic collection interfaces.

Empty Collection

Old:

```C#
List<Gift> gifts = new List<Gift>();
```

Modern:

```C#
List<Gift> gifts = [];
```

Spread

```C#
List<Gift> historical = [gift1, gift2];

List<Gift> all = [.. historical, gift3];
```

Result:

EXAMPLE / FLOW

```
gift1

gift2

gift3
```

Do Not Confuse . . Here with Range

In:

C#

```
values[1..4]
```

`..` is a range operator.

In:

C#

```
[.. existing, newValue]
```

it is a collection-expression spread element.

Same token.

Different context.

20. Read-Only Exposure in Domain Models

From Topic 16:

C#

```
public sealed class Gift
{
    private readonly List<GiftAllocation>
        _allocations = [];

    public IReadOnlyList<GiftAllocation>
        Allocations =>
        _allocations;
}
```

This is an excellent collection design pattern.

External code sees:

EXAMPLE / FLOW

```
Read access
```

The entity controls:

EXAMPLE / FLOW

```
Mutation
```

Controlled Mutation

C#

```
public void Allocate(
    Fund fund,
```

```
    decimal amount)
{
    if (
        amount <= 0m)
    {
        throw new ArgumentOutOfRangeException(
            nameof(amount));
    }

    _allocations.Add(
        new GiftAllocation(
            fund,
            amount));
}
```

Caller cannot bypass domain rules by directly calling:

```
C#
gift.Allocations.Add(...)
```

21. Collections and ASP.NET Core

Collections appear everywhere in ASP.NET Core.

Request DTO:

```
C#
public sealed record CreateGiftRequest(
    long ConstituentId,
    decimal Amount,
    IReadOnlyList<AllocationRequest>
        Allocations);
```

Response

```
C#
public sealed record GiftResponse(
    long GiftId,
    IReadOnlyList<GiftAllocationResponse>
        Allocations);
```

Service

```
C#
Task<IReadOnlyList<Gift>>
    GetGiftsAsync(
        CancellationToken cancellationToken);
```

Avoid Returning Mutable Internals

Avoid:

```
C#
public List<Gift>
    GetGifts()
```

when callers should only inspect results.

Prefer:

```C#
public IReadOnlyList<Gift>
    GetGifts()
```

or another abstraction matching the contract.

IEnumerable from Database Code

Be careful with:

```C#
IEnumerable<Gift>
```

because it doesn't inherently communicate whether data is:

```EXAMPLE / FLOW
Already materialized

Generated lazily

Backed by database query

Expensive to enumerate repeatedly
```

We'll explore query execution deeply in Topic 24 LINQ and later architecture work.

22. Thread Safety

Most ordinary generic collections such as:

```EXAMPLE / FLOW
List<T>

Dictionary<TKey,TValue>

HashSet<T>

Stack<T>
```

do not guarantee safe concurrent mutation from multiple threads.

Microsoft's `List<T>` and `Dictionary<TKey,TValue>` documentation explicitly distinguishes safe concurrent read scenarios from concurrent read/write mutation and points to concurrent collections when synchronization is required.

Example Risk

Two threads execute:

```C#
gifts.Add(
    gift);
```

against the same `List<Gift>`.

Do not assume this is safe.

Options

Depending on design:

```
EXAMPLE / FLOW
Avoid sharing mutable collection
```

or:

```
EXAMPLE / FLOW
Synchronize access
```

or use:

```
EXAMPLE / FLOW
System.Collections.Concurrent
```

such as:

```
C#
ConcurrentDictionary<TKey,TValue>
```

Concurrent and immutable collections are Topic 29, so we'll only preview them here.

Request-Scoped Objects

In ASP.NET Core, many ordinary collections are used inside one request and are not shared between threads.

Example:

```
C#
List<GiftAllocation>
```

local to request processing.

Such collections usually don't need synchronization.

Static Collection Warning

This is much more dangerous:

```
C#
public static readonly List<Gift>
    Gifts = [];
```

Now many requests may mutate the same list concurrently.

This combines the dangers of:

```
EXAMPLE / FLOW
Global mutable state

Thread safety

Lifetime

Tenant isolation
```

Avoid this design for application data.

23. Performance and Memory

Collection choice affects both runtime and memory behavior.

List

Best when:

EXAMPLE / FLOW

```
Sequential data

Indexed lookup

Appending
```

Poor when your dominant operation is:

EXAMPLE / FLOW

```
Repeated Contains
on huge lists
```

because lookup is linear.

HashSet

Better when dominant operation is:

EXAMPLE / FLOW

```
Does this value already exist?
```

Example:

C#

```
if (
    processedGiftIds.Contains(
        giftId))
```

Dictionary

Better when:

EXAMPLE / FLOW

```
Find Gift by ID
```

is dominant.

Instead of repeatedly scanning:

C#

```
List<Gift>
```

you can use:

C#

```
Dictionary<long, Gift>
```

Example

List lookup:

```C#
Gift? found = null;

foreach (
    Gift gift
    in gifts)
{
    if (
        gift.GiftId
        == targetId) { found = gift;

        break;
    }
}
```

Worst case:

```EXAMPLE / FLOW
Scan all elements
```

Dictionary Lookup

```C#
giftsById.TryGetValue(
    targetId,
    out Gift? found);
```

Typical:

```EXAMPLE / FLOW
O(1)
```

Do Not Optimize Blindly

If the collection contains:

```EXAMPLE / FLOW
5 items
```

difference may be irrelevant.

Choose based on:

```EXAMPLE / FLOW
Correct semantics first

Then measured performance
```

Memory Cost

A:

```
C#
Dictionary<TKey,TValue>
```

requires extra hashing/bucket storage.

A:

```
C#
HashSet<T>
```

also carries hash-table overhead.

A:

```
C#
List<T>
```

is typically more compact for simple sequential storage.

Use dictionary/set when their lookup semantics justify the overhead.

Capacity Planning

If importing:

```
EXAMPLE / FLOW
100,000 Gifts
```

and you know the approximate size:

```
C#
List<Gift> gifts = new(100_000);
```

may reduce resizing.

But don't trust arbitrary client-supplied capacity directly because it can trigger excessive memory allocation.

24. Security Considerations

Collections can create security problems when their size is controlled by untrusted input.

Example:

```
C#
int requestedCount = request.Count;

List<Gift> gifts = new(requestedCount);
```

If:

```
EXAMPLE / FLOW
requestedCount = 1,500,000,000
```

you may cause massive memory pressure.

Validate or clamp externally supplied sizes.

Unbounded Request Collections

API accepts:

```
C#
List<GiftRequest>
```

with no limit.

A malicious or accidental client may submit:

```
EXAMPLE / FLOW
Millions of elements
```

causing:

```
EXAMPLE / FLOW
Memory exhaustion

Long validation time

Long database operations
```

Enforce appropriate request-size/business limits.

Dictionary Collision Concerns

Hash-based collections depend on hashing.

For standard trusted platform key types such as strings, .NET provides robust implementations.

For custom key types:

```
EXAMPLE / FLOW
Implement equality correctly

Avoid pathological mutable hashing
```

Tenant Isolation

Never rely on a collection's type to provide security.

```
C#
Dictionary<long, Gift>
```

doesn't know:

```
EXAMPLE / FLOW
Tenant boundaries
```

If the dictionary contains gifts from multiple tenants:

```
C#
giftsById[
    giftId]
```

must still be protected by appropriate tenant authorization/filtering logic.

Exposing Mutable Collections

Security and correctness can also suffer when callers receive:

```C#
List<UserPermission>
```

and can alter the backing list.

Prefer controlled mutation or read-only contracts when the collection represents security-sensitive state.

25. Collection Selection Table

Requirement	Recommended Starting Point
Dynamic ordered sequence	`List<T>`
Fixed-size sequence	`T[]`
Key/value lookup	`Dictionary<TKey,TValue>`
Unique elements	`HashSet<T>`
Sorted unique elements	`SortedSet<T>`
Sorted key/value map	`SortedDictionary<TKey,TValue>`
Compact sorted map	`SortedList<TKey,TValue>`
FIFO processing	`Queue<T>`
LIFO processing	`Stack<T>`
Node-oriented sequence	`LinkedList<T>`
Enumeration-only API	`IEnumerable<T>`
Read-only count	`IReadOnlyCollection<T>`
Read-only indexed sequence	`IReadOnlyList<T>`
Mutable general collection contract	`ICollection<T>`
Mutable indexed contract	`IList<T>`

.NET's collection selection guidance makes the same fundamental distinctions among indexed sequences, key/value maps, queues, stacks, sets, and sorted structures.

Common Mistake 1 — Using List for Everything

If every lookup is:

```C#
gifts.Find(
    gift =>
        gift.GiftId
        == id);
```

and the collection is large and repeatedly queried by ID, a:

```C#
Dictionary<long, Gift>
```

may better represent the requirement.

Common Mistake 2 — Dictionary When Sequence Is Enough

If you simply need:

EXAMPLE / FLOW

```
Gift 1

Gift 2

Gift 3
```

with sequential processing, a dictionary adds unnecessary key/hash overhead.

Use:

C#

```
List<Gift>
```

Common Mistake 3 — List for Uniqueness

Bad:

C#

```
if (
    !fundIds.Contains(
        id))
{
    fundIds.Add(
        id);
}
```

repeatedly against a large list.

Better:

C#

```
HashSet<long> fundIds = [];

fundIds.Add(
    id);
```

Uniqueness is built into the data structure.

Common Mistake 4 — Assuming Dictionary Preserves Business Order

A dictionary is for:

EXAMPLE / FLOW

```
Key lookup
```

not business ordering.

If order matters:

EXAMPLE / FLOW

```
Model that requirement explicitly.
```

Don't depend on incidental enumeration behavior as a contract unless the specific API explicitly promises it.

Common Mistake 5 — Indexer for Optional Dictionary Lookup

Bad:

```C#
Gift gift = giftsById[giftId];
```

when missing keys are normal.

Better:

```C#
if (
    giftsById.TryGetValue(
        giftId,
        out Gift? gift))
{
}
```

Common Mistake 6 — ContainsKey + Indexer

Less efficient:

```C#
if (
    dictionary.ContainsKey(
        key))
{
    Value value = dictionary[key];
}
```

Better when the value is required:

```C#
if (
    dictionary.TryGetValue(
        key,
        out Value? value))
{
}
```

Common Mistake 7 — Mutable Dictionary Keys

Never mutate fields/properties that participate in:

EXAMPLE / FLOW

```
Equals

GetHashCode
```

while the object is used as a dictionary/set key.

Common Mistake 8 — Modifying Collection in foreach

Bad:

```C#
foreach (
    Gift gift
    in gifts)
{
    gifts.Remove(
        gift);
}
```

Use appropriate batch removal or index-based logic.

Common Mistake 9 — IReadOnlyList Means Immutable

It does not.

It restricts the consumer-facing API.

The underlying list may still change.

Common Mistake 10 — Returning Internal List

Bad:

```C#
public List<GiftAllocation>
    Allocations =>
        _allocations;
```

Caller can:

```C#
gift.Allocations.Clear();
```

Better:

```C#
public IReadOnlyList<GiftAllocation>
    Allocations =>
        _allocations;
```

Common Mistake 11 — Queue vs Stack Confusion

Queue:

EXAMPLE / FLOW

```
First in
First out
```

Stack:

EXAMPLE / FLOW

```
Last in
First out
```

Choose based on processing semantics.

Common Mistake 12 — HashSet for Ordered Data

TOPIC 21 · C#

`HashSet<T>` is not intended to provide sorted order.

If order matters:

EXAMPLE / FLOW

```
List<T>

SortedSet<T>

or another explicitly ordered structure
```

may fit better.

Common Mistake 13 — SortedList Is List

It isn't.

C#

```
SortedList<TKey,TValue>
```

is a sorted **key/value** collection.

Common Mistake 14 — Thread Safety Assumption

A:

C#

```
List<T>
```

or:

C#

```
Dictionary<TKey,TValue>
```

doesn't magically become safe because:

EXAMPLE / FLOW

```
It's generic.
```

Ordinary mutable collections aren't guaranteed safe for concurrent writes.

Common Mistake 15 — Excessive ToList Copies

Suppose:

C#

```
List<Gift> first = gifts.ToList();

List<Gift> second = first.ToList();

List<Gift> third = second.ToList();
```

Each may allocate/copy unnecessarily.

We'll study LINQ materialization in Topic 24.

Best Practices

Prefer generic collections.

Start with:

```C#
List<T>
```

for a general dynamic ordered sequence.

Choose:

```C#
Dictionary<TKey,TValue>
```

when key lookup is the primary requirement.

Choose:

```C#
HashSet<T>
```

when uniqueness/membership is the primary requirement.

Use:

```C#
Queue<T>
```

for FIFO workflows.

Use:

```C#
Stack<T>
```

for LIFO workflows.

Use sorted collections only when maintaining sorted order is genuinely required.

Expose:

```C#
IEnumerable<T>
```

when callers only need enumeration.

Expose:

```C#
IReadOnlyList<T>
```

for indexed read-only views.

Keep internal mutable collections private when domain invariants matter.

Prefer:

```C#
TryGetValue
```

for expected dictionary misses.

Prefer:

```C#
TryDequeue

TryPop

TryPeek
```

when empty collections are normal.

Use appropriate:

```C#
IEqualityComparer<T>
```

for domain-specific equality.

Keep dictionary/hash-set keys immutable with respect to equality and hash code.

Don't modify ordinary collections while enumerating them.

Think about:

EXAMPLE / FLOW

```
Count

Capacity

Allocation

Lookup complexity
```

when collections grow large.

Never trust external capacity/count values without validation.

Don't share mutable ordinary collections between threads without synchronization.

Don't use static mutable collections as an application database/cache without explicit concurrency, lifetime, tenant, and eviction design.

Prefer semantics before micro-optimization.

Practical Exercise 1 — Array vs List

Requirement:

EXAMPLE / FLOW

```
Exactly twelve monthly totals
```

Candidate:

```C#
decimal[] totals = new decimal[12];
```

Requirement:

EXAMPLE / FLOW

```
Unknown number of Gift allocations
```

Candidate:

```C#
List<GiftAllocation> allocations = [];
```

Practical Exercise 2 — Remove Duplicates

Given:

```C#
List<long> fundIds =
[
    10,
    20,
    10,
    30,
    20
];
```

Use:

```C#
HashSet<long> unique = new(fundIds);
```

Result:

EXAMPLE / FLOW

```
10
20
30
```

This is the cleanest solution for the interview-style “remove duplicates” problem when ordering requirements do not demand something else.

Practical Exercise 3 — Frequency Count

Input:

```C#
string[] values =
[
    "Gift",
    "Fund",
    "Gift",
    "Campaign",
    "Gift"
];
```

Count:

```C#
Dictionary<string, int> frequency = new(StringComparer.Ordinal);

foreach (
    string value
    in values)
{
    if (
        frequency.TryGetValue(
```

```
            value,
            out int count))
    {
        frequency[value] = count + 1;
    }
    else
    {
        frequency[value] = 1;
    }
}
```

Result:

EXAMPLE / FLOW

```
Gift     → 3

Fund     → 1

Campaign → 1
```

Practical Exercise 4 — Gift Lookup

C#

```
Dictionary<long, Gift> gifts = new();

foreach (
    Gift gift
    in source)
{
    gifts[
        gift.GiftId] = gift;
}
```

Then:

C#

```
if (
    gifts.TryGetValue(
        giftId,
        out Gift? gift))
{
    Process(
        gift);
}
```

Practical Exercise 5 — Queue

C#

```
Queue<Gift> pending = new();

pending.Enqueue(
    gift1);

pending.Enqueue(
    gift2);

Gift next = pending.Dequeue();
```

Result:

because the queue is FIFO.

Practical Exercise 6 — Stack

```C#
Stack<string> changes = new();

changes.Push(
    "Created");

changes.Push(
    "Changed Amount");

changes.Push(
    "Changed Campaign");
```

Undo:

```C#
string lastChange = changes.Pop();
```

Result:

```
EXAMPLE / FLOW
Changed Campaign
```

Practical Exercise 7 — Duplicate Allocation Protection

```C#
HashSet<long> funds = [];

public void AddFund(
    long fundId)
{
    if (
        !funds.Add(
            fundId))
    {
        throw new InvalidOperationException(
            "Fund has already been allocated.");
    }
}
```

Practical Exercise 8 — Read-Only Domain Collection

```C#
public sealed class Gift
{
    private readonly List<GiftAllocation>
        _allocations = [];

    public IReadOnlyList<GiftAllocation>
        Allocations =>
        _allocations;
```

```
    public void Allocate(
        GiftAllocation allocation)
    {
        ArgumentNullException.ThrowIfNull(
            allocation);

        _allocations.Add(
            allocation);
    }
}
```

Practical Exercise 9 — Case-Insensitive Codes

C#

```
HashSet<string> campaignCodes = new(StringComparer.OrdinalIgnoreCase);

campaignCodes.Add(
    "YEAR-END");

bool added = campaignCodes.Add("year-end");
```

Result:

EXAMPLE / FLOW

```
false
```

Practical Exercise 10 — Sorted Values

C#

```
SortedSet<decimal> amounts =
[
    500m,
    100m,
    250m,
    100m
];
```

Logical contents:

EXAMPLE / FLOW

```
100
250
500
```

Sorted and unique.

Practical Exercise 11 — Remove Cancelled Gifts

C#

```
gifts.RemoveAll(
    gift =>
        gift.Status
        == GiftStatus.Cancelled);
```

Prefer this to removing from the list inside `foreach`.

Practical Exercise 12 — Choosing Collection

Requirement:

EXAMPLE / FLOW

```
Lookup Constituents repeatedly
by ConstituentId
```

Choose:

C#

```
Dictionary<long, Constituent>
```

Requirement:

EXAMPLE / FLOW

```
Store ordered GiftAllocations
and allow duplicates if business permits
```

Choose:

C#

```
List<GiftAllocation>
```

Requirement:

EXAMPLE / FLOW

```
Track unique processed GiftIds
```

Choose:

C#

```
HashSet<long>
```

Interview Questions

1. What is a collection in C#?

A collection is a data structure that stores multiple elements and provides operations appropriate to those elements, such as enumeration, insertion, removal, lookup, ordering, or uniqueness.

2. Why are generic collections preferred over non-generic collections?

Generic collections preserve compile-time element types, reduce casting errors, and avoid unnecessary boxing in many value-type scenarios.

For example:

C#

```
List<int>
```

is safer than an object-based `ArrayList`.

3. What is the difference between an array and `List<T>`?

An array has fixed length.

`List<T>` has a dynamic logical size and can grow its backing capacity.

Both support indexed access.

4. What is IEnumerable<T>?

It represents a sequence that can be enumerated.

Its primary contract is iteration, not mutation, indexing, or random access.

5. What is the difference between IEnumerable<T>, ICollection<T>, and IList<T>?

Conceptually:

EXAMPLE / FLOW

```
IEnumerable<T>
    →
enumeration

ICollection<T>
    →
enumeration + count + basic mutation

IList<T>
    →
collection + ordered index access
```

`ICollection<T>` exposes operations such as `Count`, `Add`, `Remove`, and `Contains`.

6. What is the difference between List<T> Count and Capacity?

`Count` is the number of elements currently stored.

`Capacity` is the amount of internal storage currently available before another growth operation is required.

7. What is the typical complexity of adding to List<T>?

Appending is amortized:

EXAMPLE / FLOW

```
O(1)
```

but an insertion that triggers backing-array growth can require:

EXAMPLE / FLOW

```
O(n)
```

to allocate and copy elements.

8. Why is inserting near the beginning of a List<T> expensive?

Because later elements generally need to be shifted to make room.

Therefore insertion/removal in the middle is typically:

EXAMPLE / FLOW

```
O(n)
```

9. When should you use `Dictionary<TKey,TValue>`?

Use it when the dominant requirement is:

EXAMPLE / FLOW

```
Find a value efficiently
using a unique key.
```

Example:

C#

```
Dictionary<long, Gift>
```

for:

EXAMPLE / FLOW

```
GiftId → Gift
```

Dictionary lookup is typically close to O(1).

10. What is the difference between `Dictionary.Add` and the dictionary indexer?

`Add` requires the key to be absent and throws on a duplicate key.

The indexer:

C#

```
dictionary[key] = value;
```

adds if absent and replaces the value if the key already exists.

11. Why is `TryGetValue` often preferable to `ContainsKey` followed by the indexer?

If you need the value, `TryGetValue` performs lookup and returns the value in one operation.

It also avoids exception-based control flow for expected missing keys.

12. What makes a good dictionary key?

A good key has stable equality and hash behavior while stored.

Common choices include:

EXAMPLE / FLOW

```
int

long

Guid

string

immutable value objects
```

Mutating fields that affect a key's hash code can make the entry difficult or impossible to locate.

13. What is `HashSet<T>` used for?

`HashSet<T>` represents a set of unique elements.

It is useful for:

EXAMPLE / FLOW

```
Uniqueness

Fast membership tests

Union

Intersection

Difference
```

and implements `ISet<T>`.

14. What is the difference between `List<T>` and `HashSet<T>`?

`List<T>`:

EXAMPLE / FLOW

```
Ordered

Duplicates allowed

Indexable
```

`HashSet<T>`:

EXAMPLE / FLOW

```
Unique elements

No indexed position semantics

Fast membership lookup
```

15. What is the difference between `Queue<T>` and `Stack<T>`?

`Queue<T>` uses:

EXAMPLE / FLOW

```
FIFO
```

while `Stack<T>` uses:

EXAMPLE / FLOW

```
LIFO
```

Queue operations include `Enqueue`, `Dequeue`, and `Peek`; stack operations include `Push`, `Pop`, and `Peek`.

16. When would you use `LinkedList<T>` instead of `List<T>`?

Consider `LinkedList<T>` when insertion/removal around already-known nodes is central and random indexed access isn't needed.

For ordinary sequential application data, `List<T>` is usually simpler.

17. What is the difference between `HashSet<T>` and `SortedSet<T>`?

`HashSet<T>` is hash-based and optimized for fast set membership without sorted-order semantics.

`SortedSet<T>` maintains elements in sorted order and uses tree-like O(log n) operations.

18. What is the difference between `SortedDictionary<TKey,TValue>` and `SortedList<TKey,TValue>`?

TOPIC 21 · C#

Both maintain keys in sorted order and provide O(log n) retrieval.

`SortedDictionary` generally provides better insertion/removal for changing unsorted data, while `SortedList` uses less memory and has O(n) insertion/removal because array entries may need shifting.

19. What is `IReadOnlyList<T>`?

It provides:

EXAMPLE / FLOW

```
Enumeration

Count

Read-only index access
```

but doesn't expose standard mutation operations.

It does **not** guarantee that the underlying collection or its elements are deeply immutable.

20. Can you safely modify a `List<T>` inside a `foreach` over that same list?

Normally no.

Structural modification invalidates ordinary enumeration and commonly results in `InvalidOperationException`.

Use operations such as `RemoveAll`, create a copy, or use an appropriate index-based algorithm.

21. How do custom equality comparers affect collections?

`Dictionary<TKey,TValue>` and `HashSet<T>` can receive an:

C#

```
IEqualityComparer<T>
```

that defines equality and hashing.

Example:

C#

```
StringComparer.OrdinalIgnoreCase
```

supports case-insensitive string keys/set values.

22. Are `List<T>` and `Dictionary<TKey,TValue>` thread-safe?

Their instance members don't guarantee safe concurrent mutation.

If multiple threads need coordinated reads and writes, use synchronization or a collection designed for concurrency.

23. What are collection expressions?

Modern C# allows target-typed collection syntax:

C#

```
List<int> values = [1, 2, 3];
```

It also supports spread elements:

```C#
[.. existing, newValue]
```

for supported collection targets.

24. What collection would you use to remove duplicates?

Typically:

```C#
HashSet<T>
```

because uniqueness is intrinsic to the set.

Example:

```C#
HashSet<long> unique = new(ids);
```

If original ordering must be preserved, additional logic may be required.

25. How do you choose the correct collection?

Choose based on the operations the application actually needs:

EXAMPLE / FLOW

```
Dynamic ordered sequence
    →
List<T>

Key lookup
    →
Dictionary<TKey,TValue>

Unique membership
    →
HashSet<T>

FIFO
    →
Queue<T>

LIFO
    →
Stack<T>

Sorted uniqueness
    →
SortedSet<T>
```

Collection choice should reflect semantics first, followed by measured performance requirements.

Quick Revision

Collection	Main Purpose	Duplicates	Ordered/Sorted	Typical Lookup
`T[]`	Fixed sequence	Yes	Insertion order	Index O(1)

Collection	Main Purpose	Duplicates	Ordered/Sorted	Typical Lookup
`List<T>`	Dynamic sequence	Yes	Insertion/index order	Index O(1), search O(n)
`Dictionary<TKey,TValue>`	Key/value mapping	Keys no	Don't use as business-order contract	Avg O(1) key lookup
`HashSet<T>`	Unique values	No	Not sorted	Avg O(1) membership
`Queue<T>`	FIFO	Yes	Queue order	Front
`Stack<T>`	LIFO	Yes	Stack order	Top
`LinkedList<T>`	Node-oriented sequence	Yes	Sequence order	Traversal
`SortedSet<T>`	Sorted unique values	No	Sorted	O(log n)
`SortedDictionary<TKey,TValue>`	Sorted map	Keys no	Sorted by key	O(log n)
`SortedList<TKey,TValue>`	Compact sorted map	Keys no	Sorted by key	O(log n) lookup

Complete Fundraising Example

Consider a `Gift` that:

EXAMPLE / FLOW

```
Has ordered allocations

Must prevent duplicate Fund allocation

Needs fast lookup by FundId
```

Different collections solve different parts of the problem.

Fund

C#

```
public sealed class Fund
{
    public Fund(
        long fundId,
        string name)
    {
        if (
            fundId <= 0)
        {
            throw new ArgumentOutOfRangeException(
                nameof(fundId));
        }

        ArgumentException.ThrowIfNullOrWhiteSpace(
            name);

        FundId = fundId;

        Name = name;
    }
```

```
    public long FundId
    {
        get;
    }

    public string Name
    {
        get;
    }
}
```

GiftAllocation

C#

```
public sealed class GiftAllocation
{
    internal GiftAllocation(
        Fund fund,
        decimal amount)
    {
        ArgumentNullException.ThrowIfNull(
            fund);

        if (
            amount <= 0m)
        {
            throw new ArgumentOutOfRangeException(
                nameof(amount));
        }

        Fund = fund;

        Amount = amount;
    }

    public Fund Fund
    {
        get;
    }

    public decimal Amount
    {
        get;
    }
}
```

Gift

C#

```
public sealed class Gift
{
    private readonly List<GiftAllocation>
        _allocations = [];

    private readonly Dictionary<
        long,
        GiftAllocation>
        _allocationsByFundId = new();

    public Gift(
```

```
        long giftId,
        decimal totalAmount)
    {
        if (
            giftId <= 0)
        {
            throw new ArgumentOutOfRangeException(
                nameof(giftId));
        }

        if (
            totalAmount <= 0m)
        {
            throw new ArgumentOutOfRangeException(
                nameof(totalAmount));
        }

        GiftId = giftId;

        TotalAmount = totalAmount;
    }

    public long GiftId
    {
        get;
    }

    public decimal TotalAmount
    {
        get;
    }

    public IReadOnlyList<GiftAllocation>
        Allocations =>
        _allocations;

    public decimal AllocatedAmount
    {
        get
        {
            decimal total = 0m;

            foreach (
                GiftAllocation allocation
                in _allocations)
            {
                total += allocation.Amount;
            }

            return total;
        }
    }

    public decimal UnallocatedAmount
        =>
        TotalAmount
        -
        AllocatedAmount;

    public void Allocate(
        Fund fund,
        decimal amount)
    {
        ArgumentNullException.ThrowIfNull(
            fund);
```

```
        if (
            amount <= 0m)
        {
            throw new ArgumentOutOfRangeException(
                nameof(amount));
        }

        if (
            amount
            >
            UnallocatedAmount)
        {
            throw new InvalidOperationException(
                "Allocation exceeds the remaining gift amount.");
        }

        if (
            _allocationsByFundId.ContainsKey(
                fund.FundId))
        {
            throw new InvalidOperationException(
                "The gift already contains an allocation for this fund.");
        }

        GiftAllocation allocation = new(fund, amount);

        _allocations.Add(
            allocation);

        _allocationsByFundId.Add(
            fund.FundId,
            allocation);
    }

    public bool TryGetAllocation(
        long fundId,
        out GiftAllocation? allocation)
    {
        return _allocationsByFundId
            .TryGetValue(
                fundId,
                out allocation);
    }
}
```

Why Two Collections?

The list:

C#

```
List<GiftAllocation>
```

provides:

EXAMPLE / FLOW

```
Ordered allocation sequence
```

The dictionary:

provides:

EXAMPLE / FLOW
Fast lookup by FundId

This demonstrates an important design lesson:

> **Sometimes one object legitimately maintains more than one data structure because different access patterns have different requirements.**

But this introduces synchronization responsibility.

Whenever:

EXAMPLE / FLOW
_allocations

changes, you must keep:

EXAMPLE / FLOW
_allocationsByFundId

consistent.

Alternative with HashSet

If all you need is duplicate detection:

```C#
private readonly HashSet<long>
    _allocatedFundIds = [];
```

Then:

```C#
if (
    !_allocatedFundIds.Add(
        fund.FundId))
{
    throw new InvalidOperationException(
        "Fund already allocated.");
}
```

This may be simpler than a dictionary if you never need:

EXAMPLE / FLOW
FundId → GiftAllocation

lookup.

Collection Design Principle

Ask:

EXAMPLE / FLOW

What operations are required?

Not:

EXAMPLE / FLOW

What collection do I usually use?

Fundraising Batch Processing Example

Pending gifts:

C#

```
Queue<Gift> pendingGifts = new();
```

Add work:

C#

```
pendingGifts.Enqueue(
    gift1);

pendingGifts.Enqueue(
    gift2);
```

Process in arrival order:

C#

```
while (
    pendingGifts.TryDequeue(
        out Gift? gift))
{
    ProcessGift(
        gift);
}
```

Flow:

EXAMPLE / FLOW

```
Gift 1
    ↓
Gift 2
    ↓
Gift 3
```

FIFO.

Processed Gift IDs

C#

```
HashSet<long> processedGiftIds = [];
```

Before processing:

C#

```
if (
    !processedGiftIds.Add(
        gift.GiftId))
{
```

```
    continue;
}
```

This guarantees:

EXAMPLE / FLOW

```
Process each GiftId once
```

inside this in-memory operation.

Gift Lookup Cache

C#

```
Dictionary<long, Gift> giftsById = new();
```

Lookup:

C#

```
if (
    giftsById.TryGetValue(
        giftId,
        out Gift? gift))
{
    return gift;
}
```

Remember:

EXAMPLE / FLOW

```
In-memory cache
```

introduces additional concerns such as:

EXAMPLE / FLOW

```
Lifetime

Thread safety

Eviction

Tenant isolation

Staleness
```

so don't turn an ordinary dictionary into a global production cache without intentional design.

Final Collection Hierarchy Mental Model

EXAMPLE / FLOW

```
Need enumeration
    →
IEnumerable<T>
```

EXAMPLE / FLOW

```
Need count + basic mutation
    →
ICollection<T>
```

EXAMPLE / FLOW

```
Need indexed mutation
    →
IList<T>
```

EXAMPLE / FLOW

```
Need read-only count
    →
IReadOnlyCollection<T>
```

EXAMPLE / FLOW

```
Need read-only index
    →
IReadOnlyList<T>
```

Final Concrete-Type Mental Model

EXAMPLE / FLOW

```
Sequence
    →
List<T>
```

EXAMPLE / FLOW

```
Key → Value
    →
Dictionary<TKey,TValue>
```

EXAMPLE / FLOW

```
Unique membership
    →
HashSet<T>
```

EXAMPLE / FLOW

```
FIFO workflow
    →
Queue<T>
```

EXAMPLE / FLOW

```
LIFO workflow
    →
Stack<T>
```

EXAMPLE / FLOW

```
Sorted unique values
    →
SortedSet<T>
```

EXAMPLE / FLOW

```
Sorted key/value mapping
    →
SortedDictionary<TKey,TValue>
```

Final Performance Mental Model

EXAMPLE / FLOW

```
List<T>
    →
Fast indexing

Slow arbitrary lookup
```

EXAMPLE / FLOW

```
Dictionary<TKey,TValue>
    →
Fast key lookup
```

EXAMPLE / FLOW

```
HashSet<T>
    →
Fast membership
```

EXAMPLE / FLOW

```
Sorted structures
    →
Ordering maintained
at additional logarithmic/update cost
```

Final End-to-End Concept Flow

EXAMPLE / FLOW

```
Collection requirement
        ↓
Do I need fixed size?
    ├─ Yes → Array
    └─ No
        ↓
Need unique key/value lookup?
    ├─ Yes → Dictionary
    └─ No
        ↓
Need uniqueness only?
    ├─ Yes → HashSet
    └─ No
        ↓
Need FIFO?
    ├─ Yes → Queue
    └─ No
        ↓
Need LIFO?
    ├─ Yes → Stack
    └─ No
        ↓
Need sorted behavior?
    ├─ Yes → Sorted collection
    └─ No  → List<T>
```

Then decide what consumers should see:

EXAMPLE / FLOW

```
Full mutation?
    →
Concrete collection / ICollection / IList
```

EXAMPLE / FLOW

```
Read-only?
    →
IReadOnlyCollection / IReadOnlyList
```

EXAMPLE / FLOW

```
Enumeration only?
    →
IEnumerable<T>
```

The core lesson is:

> **Choose collections according to semantics and access patterns. `List<T>` is an excellent default for dynamic ordered data, but `Dictionary<TKey,TValue>` is better for key lookup, `HashSet<T>` for uniqueness, `Queue<T>` for FIFO processing, `Stack<T>` for LIFO processing, and sorted collections when order must be maintained automatically. Expose only the operations consumers need, keep domain collections encapsulated, use stable equality for hash-based collections, and account for complexity, capacity, concurrency, and security when collections become large or shared.**

For the Fundraising Platform, common choices are:

EXAMPLE / FLOW

```
Gift.Allocations
    →
List<GiftAllocation>
internally
+
IReadOnlyList<GiftAllocation>
publicly

GiftId → Gift
    →
Dictionary<long, Gift>

Processed GiftIds
    →
HashSet<long>

Receipt work
    →
Queue<GiftReceipt>

Undo/history
    →
Stack<GiftChange>
```

Next roadmap topic: Topic 22 — Delegates, Action, Func and Predicate.